Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2023

#RANKED 52 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED GRADE 'A' BY NAAC

PERFECT SCORE OF 150/150 AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Elements of AIML

Unit 02 – Lecture 03: Clause Form, Resolution & Unification

Tofik Ali

School of Computer Science, UPES Dehradun

February 28, 2026

Topic focus: **automated inference** via refutation (resolution)

Repository: `https://github.com/tali7c/Elements-of-AIML`

Sources: Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 7–9 (Logical Agents, First-Order Logic, Inference).;Huth and Ryan, *Logic in Computer Science* (2e), selected sections on propositional and first-order logic.;Genesereth and Nilsson, *Logical Foundations of Artificial Intelligence*, selected chapters.

Quick Links

Clause Form Resolution (PL) Resolution (FOL) Unification Summary

Agenda

Clause form (CNF)

Resolution in propositional logic

Resolution in first-order logic

Unification

Summary

Learning Outcomes

By the end of this lecture, you should be able to:

- Convert simple propositional sentences to **CNF / clause form**.

Learning Outcomes

By the end of this lecture, you should be able to:

- Convert simple propositional sentences to **CNF / clause form**.
- Explain **resolution** as a refutation proof method.

Learning Outcomes

By the end of this lecture, you should be able to:

- Convert simple propositional sentences to **CNF / clause form**.
- Explain **resolution** as a refutation proof method.
- Apply resolution to small propositional knowledge bases.

Learning Outcomes

By the end of this lecture, you should be able to:

- Convert simple propositional sentences to **CNF / clause form**.
- Explain **resolution** as a refutation proof method.
- Apply resolution to small propositional knowledge bases.
- Explain **unification** and why it is needed for FOL resolution.

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)
- $\models$: semantic entailment (true in all models)

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)
- $\models$: semantic entailment (true in all models)
- $\vdash$: syntactic derivation (provable in a proof system)

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)
- $\models$: semantic entailment (true in all models)
- $\vdash$: syntactic derivation (provable in a proof system)
- $\neg, \wedge, \vee$: NOT, AND, OR

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)
- $\models$: semantic entailment (true in all models)
- $\vdash$: syntactic derivation (provable in a proof system)
- $\neg, \wedge, \vee$: NOT, AND, OR
- CNF: conjunctive normal form; DNF: disjunctive normal form

Abbreviations & Symbols

- KB: knowledge base (set of sentences/facts/rules)
- α: query (goal sentence)
- $\models$: semantic entailment (true in all models)
- $\vdash$: syntactic derivation (provable in a proof system)
- $\neg, \wedge, \vee$: NOT, AND, OR
- CNF: conjunctive normal form; DNF: disjunctive normal form
- $\square$: empty clause (contradiction); θ: substitution; MGU: most general unifier

Refutation View of Entailment

Key equivalence (idea used by SAT/resolution):

$$KB \models \alpha \quad \Leftrightarrow \quad KB \wedge \neg\alpha \text{ is unsatisfiable}$$

So to prove $KB \models \alpha$, we:

1. add $\neg\alpha$ to KB,
2. derive a contradiction (empty clause) using resolution.

Clause Form (CNF)

- CNF: **AND** of **OR** clauses:

$$(a \vee b \vee \neg c) \wedge (\neg a \vee d)$$

Clause Form (CNF)

- CNF: **AND** of **OR** clauses:

$$(a \vee b \vee \neg c) \wedge (\neg a \vee d)$$

- A **literal** is a variable or its negation (e.g., p or $\neg p$).

Clause Form (CNF)

- CNF: **AND** of **OR** clauses:

$$(a \vee b \vee \neg c) \wedge (\neg a \vee d)$$

- A **literal** is a variable or its negation (e.g., p or $\neg p$).
- A **clause** is a disjunction of literals.

CNF Conversion Recipe (PL)

Convert any propositional formula into CNF using these steps:

1. Eliminate $\leftrightarrow$ and $\rightarrow$

CNF is a **representation choice** that makes automated inference systematic.

CNF Conversion Recipe (PL)

Convert any propositional formula into CNF using these steps:

1. Eliminate $\leftrightarrow$ and $\rightarrow$
2. Push $\neg$ inward (De Morgan + double negation)

CNF is a **representation choice** that makes automated inference systematic.

CNF Conversion Recipe (PL)

Convert any propositional formula into CNF using these steps:

1. Eliminate $\leftrightarrow$ and $\rightarrow$
2. Push $\neg$ inward (De Morgan + double negation)
3. Distribute $\vee$ over $\wedge$

CNF is a **representation choice** that makes automated inference systematic.

CNF Conversion Recipe (PL)

Convert any propositional formula into CNF using these steps:

1. Eliminate $\leftrightarrow$ and $\rightarrow$
2. Push $\neg$ inward (De Morgan + double negation)
3. Distribute $\vee$ over $\wedge$
4. Split the final $\wedge$ into a set of clauses

CNF is a **representation choice** that makes automated inference systematic.

CNF Conversion (Mini Example)

Convert: $(p \to q) \land p$

- Eliminate implication: $(\neg p \lor q) \land p$

CNF Conversion (Mini Example)

Convert: $(p \to q) \land p$

- Eliminate implication: $(\neg p \lor q) \land p$
- Already in CNF:

$$(\neg p \lor q) \land (p)$$

CNF Conversion (Needs Distribution)

Convert: $p \to (q \wedge r)$

- Eliminate implication:

$$\neg p \vee (q \wedge r)$$

CNF Conversion (Needs Distribution)

Convert: $p \to (q \wedge r)$

- Eliminate implication:

$$\neg p \vee (q \wedge r)$$

- Distribute $\vee$ over $\wedge$:

$$(\neg p \vee q) \wedge (\neg p \vee r)$$

CNF Conversion (Needs Distribution)

Convert: $p \to (q \wedge r)$

- Eliminate implication:

$$\neg p \vee (q \wedge r)$$

- Distribute $\vee$ over $\wedge$:

$$(\neg p \vee q) \wedge (\neg p \vee r)$$

- Clauses: $(\neg p \vee q)$ and $(\neg p \vee r)$

Resolution Rule (PL)

From two clauses:

$$(A \vee p), \quad (B \vee \neg p)$$

we can infer:

$$(A \vee B)$$

Intuition: if either p is true or false, one of the two clauses forces the remaining parts.

Worked Example (PL): $(p \to q), p \models q$

- $KB = \{p \to q,\ p\}$, query $\alpha = q$

Therefore $KB \models q$.

Worked Example (PL): $(p \to q), p \models q$

- $KB = \{p \to q,\ p\}$, query $\alpha = q$
- Add $\neg q$ and convert to CNF:

$$(\neg p \lor q),\ (p),\ (\neg q)$$

Therefore $KB \models q$.

Worked Example (PL): $(p \to q), p \models q$

- $KB = \{p \to q,\ p\}$, query $\alpha = q$
- Add $\neg q$ and convert to CNF:

$$(\neg p \vee q),\ (p),\ (\neg q)$$

- Resolve $(\neg p \vee q)$ with (p) gives (q)

Therefore $KB \models q$.

Worked Example (PL): $(p \to q), p \models q$

- $KB = \{p \to q,\ p\}$, query $\alpha = q$
- Add $\neg q$ and convert to CNF:

$$(\neg p \vee q),\ (p),\ (\neg q)$$

- Resolve $(\neg p \vee q)$ with (p) gives (q)
- Resolve (q) with $(\neg q)$ gives **empty clause** $\square$

Therefore $KB \models q$.

Worked Example (PL): $(p \to q), (q \to r), p \models r$

- Add $\neg r$ and convert to CNF:

$$(\neg p \lor q),\ (\neg q \lor r),\ (p),\ (\neg r)$$

This shows **entailment by contradiction**.

Worked Example (PL): $(p \to q), (q \to r), p \models r$

- Add $\neg r$ and convert to CNF:

$$(\neg p \lor q),\ (\neg q \lor r),\ (p),\ (\neg r)$$

- Resolve $(\neg p \lor q)$ with (p) gives (q)

This shows **entailment by contradiction**.

Worked Example (PL): $(p \to q), (q \to r), p \models r$

- Add $\neg r$ and convert to CNF:

$$(\neg p \lor q),\ (\neg q \lor r),\ (p),\ (\neg r)$$

- Resolve $(\neg p \lor q)$ with (p) gives (q)
- Resolve $(\neg q \lor r)$ with (q) gives (r)

This shows **entailment by contradiction**.

Worked Example (PL): $(p \to q), (q \to r), p \models r$

- Add $\neg r$ and convert to CNF:

$$(\neg p \lor q),\ (\neg q \lor r),\ (p),\ (\neg r)$$

- Resolve $(\neg p \lor q)$ with (p) gives (q)
- Resolve $(\neg q \lor r)$ with (q) gives (r)
- Resolve (r) with $(\neg r)$ gives $\square$

This shows **entailment by contradiction**.

FOL Resolution: What's New?

- Variables and quantifiers

FOL Resolution: What's New?

- Variables and quantifiers
- Need to convert to a clause form suitable for resolution

FOL Resolution: What's New?

- Variables and quantifiers
- Need to convert to a clause form suitable for resolution
- When literals "almost match", we use **unification** to make them match

FOL Clause Form Pipeline (High-Level)

1. Remove $\leftrightarrow, \rightarrow$

The output is a **set of clauses** used by resolution.

FOL Clause Form Pipeline (High-Level)

1. Remove $\leftrightarrow, \rightarrow$
2. Push $\neg$ inward

The output is a **set of clauses** used by resolution.

FOL Clause Form Pipeline (High-Level)

1. Remove $\leftrightarrow, \rightarrow$
2. Push $\neg$ inward
3. Standardize variables apart (rename to avoid clashes)

The output is a **set of clauses** used by resolution.

FOL Clause Form Pipeline (High-Level)

1. Remove $\leftrightarrow, \rightarrow$
2. Push $\neg$ inward
3. Standardize variables apart (rename to avoid clashes)
4. Skolemize $\exists$ (replace with new constants/functions)

The output is a **set of clauses** used by resolution.

FOL Clause Form Pipeline (High-Level)

1. Remove $\leftrightarrow, \rightarrow$
2. Push $\neg$ inward
3. Standardize variables apart (rename to avoid clashes)
4. Skolemize $\exists$ (replace with new constants/functions)
5. Drop $\forall$ (implicit in clauses), distribute, split into clauses

The output is a **set of clauses** used by resolution.

Skolemization (Intuition)

- $\exists$ means "there exists something" — introduce a **new symbol** for it.

Skolemization (Intuition)

- $\exists$ means "there exists something" — introduce a **new symbol** for it.
- Example:

$$\forall x\ \exists y\ Likes(x, y)$$

becomes

$$\forall x\ Likes(x, f(x))$$

Skolemization (Intuition)

- $\exists$ means "there exists something" — introduce a **new symbol** for it.
- Example:

$$\forall x\ \exists y\ Likes(x, y)$$

becomes

$$\forall x\ Likes(x, f(x))$$

- f is a **Skolem function** (depends on x).

Classic FOL Example: Socrates is Mortal

Knowledge:

- $\forall x\ Human(x) \rightarrow Mortal(x)$
- $Human(Socrates)$

Query: $Mortal(Socrates)$ Refutation: add $\neg Mortal(Socrates)$ and resolve to contradiction.

Socrates: Clauses + One Resolution Step

CNF (clause form):

- $\neg Human(x) \lor Mortal(x)$

Resolve $Human(Socrates)$ with $\neg Human(x) \lor Mortal(x)$ using $\{x/Socrates\}$ to infer $Mortal(Socrates)$, then derive $\square$ with $\neg Mortal(Socrates)$.

Socrates: Clauses + One Resolution Step

CNF (clause form):

- $\neg Human(x) \lor Mortal(x)$
- $Human(Socrates)$

Resolve $Human(Socrates)$ with $\neg Human(x) \lor Mortal(x)$ using $\{x/Socrates\}$ to infer $Mortal(Socrates)$, then derive $\square$ with $\neg Mortal(Socrates)$.

Socrates: Clauses + One Resolution Step

CNF (clause form):

- $\neg Human(x) \lor Mortal(x)$
- $Human(Socrates)$
- $\neg Mortal(Socrates)$ (negated query)

Resolve $Human(Socrates)$ with $\neg Human(x) \lor Mortal(x)$ using $\{x/Socrates\}$ to infer $Mortal(Socrates)$, then derive $\square$ with $\neg Mortal(Socrates)$.

What is Unification?

Unification finds a substitution that makes two expressions identical.

Example

Unify $Likes(x, AI)$ and $Likes(Ali, AI)$

$$\Rightarrow \{x/Ali\}$$

We prefer the **most general unifier (MGU)**.

Occurs-Check (Why some unifications fail)

Trying to unify x with $f(x)$ should fail:

$$x = f(x)$$

because it would require an infinite term $x = f(f(f(\dots)))$. This check avoids inconsistent substitutions.

Unification Steps (Algorithm Idea)

To unify two expressions, repeatedly:

1. if they are identical, continue

Output is a substitution θ; the best one is the **MGU**.

Unification Steps (Algorithm Idea)

To unify two expressions, repeatedly:

1. if they are identical, continue
2. if one side is a variable, substitute it (if occurs-check passes)

Output is a substitution θ; the best one is the **MGU**.

Unification Steps (Algorithm Idea)

To unify two expressions, repeatedly:

1. if they are identical, continue
2. if one side is a variable, substitute it (if occurs-check passes)
3. if both are functions/predicates, unify their corresponding arguments

Output is a substitution θ; the best one is the **MGU**.

Unification Steps (Algorithm Idea)

To unify two expressions, repeatedly:

1. if they are identical, continue
2. if one side is a variable, substitute it (if occurs-check passes)
3. if both are functions/predicates, unify their corresponding arguments
4. otherwise, fail (no unifier exists)

Output is a substitution θ; the best one is the **MGU**.

Unification Example (MGU)

Unify:

$$Knows(x, f(y), y) \text{ and } Knows(Ali, f(z), z)$$

- $x \mapsto Ali$

MGU: $\{x/Ali,\ y/z\}$.

Unification Example (MGU)

Unify:

$$Knows(x, f(y), y) \text{ and } Knows(Ali, f(z), z)$$

- $x \mapsto Ali$
- need $f(y) = f(z)$ so $y \mapsto z$

MGU: $\{x/Ali,\ y/z\}$.

Summary

- To prove $KB \models \alpha$, check unsatisfiability of $KB \wedge \neg\alpha$.
- Resolution works on CNF (clauses).
- FOL resolution needs unification to match literals with variables.

Sources: Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 7–9 (Logical Agents, First-Order Logic, Inference).;Huth and Ryan, *Logic in Computer Science* (2e), selected sections on propositional and first-order logic.;Genesereth and Nilsson, *Logical Foundations of Artificial Intelligence*, selected chapters.

Exit Questions

1. Why do we convert to CNF before resolution?
2. State the resolution rule for propositional logic.
3. Explain unification with an example substitution.
4. Why does the occurs-check matter?